

Reinforcement Learning

Policy Search – TRPO & PPO

Naeemullah Khan

naeemullah.khan@kaust.edu.sa

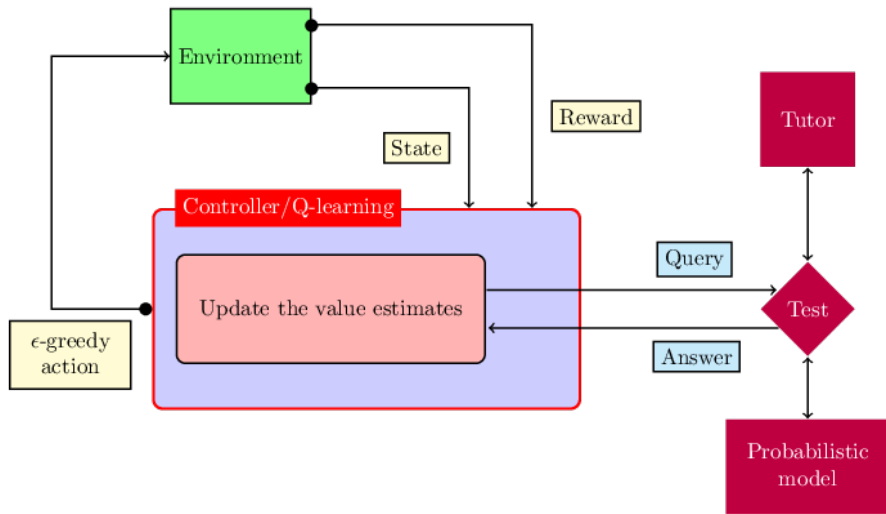


جامعة الملك عبد الله
للعلوم والتقنية
King Abdullah University of
Science and Technology



LMH
Lady Margaret Hall
University of Oxford

June 18, 2026



1. Recap: Policy Gradient
2. Learning Outcomes
3. Motivation
4. Types of Policies
 1. Example: Aliased Grid world
5. Limitations of policy gradients
6. Importance Sampling
7. Relative Policy Performance Identity
8. Choosing a Step Size for Policy Gradients
9. Trust Region Policy Optimization (TRPO)
10. Proximal Policy Optimization (PPO)
11. Summary
12. References

Policy Gradient: **Recap**

- ▶ Learning the exact Q-value for every (state, action) pair is challenging in high-dimensional spaces.
- ▶ Instead, we can directly learn a policy that maximizes expected reward.
- ▶ Policy parameters can be optimized using gradient ascent.
- ▶ However, policy gradients can suffer from high variance; various strategies exist to address this.
- ▶ Actor-Critic methods combine policy gradients and value-based methods by training both an actor (the policy) and a critic (the value function).
- ▶ The actor selects actions, while the critic evaluates the actions and guides the actor's learning.

- ▶ Understand the challenges of vanilla policy gradients.
- ▶ Explain the motivation and theory behind Trust Region Policy Optimization (TRPO).
- ▶ Describe and implement Proximal Policy Optimization (PPO).
- ▶ Identify the trade-offs between stability and performance in modern policy optimization algorithms.
- ▶ Recognize applications and evaluate limitations of TRPO and PPO.

- ▶ **Vanilla policy gradient methods** (e.g., REINFORCE, A2C) suffer from:
 - High variance in gradient estimates
 - Unstable policy updates
 - Requirement for small learning rates
 - Large policy updates may collapse performance
- ▶ **Goal:** Achieve stable and efficient policy updates while ensuring policy improvement

Policy Search: **Types of Policies**

- ▶ Policy π determines how the agent chooses actions

- ▶ Deterministic Policy:

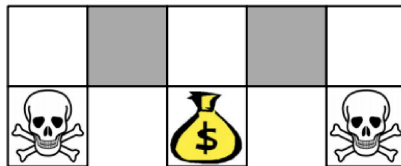
$$\pi(s) = a$$

- ▶ Stochastic Policy:

$$\pi(a|s) = Pr(a_t = a | s_t = s)$$

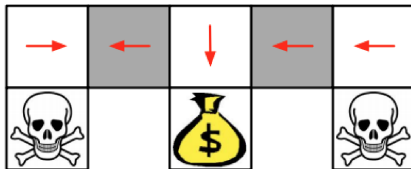
- ▶ So far have focused on deterministic policies or ϵ -greedy policies
- ▶ ϵ -greedy policies are also near deterministic as we decrease the value of epsilon with training
- ▶ Is deterministic policy optimal?

Example: Aliased Grid world



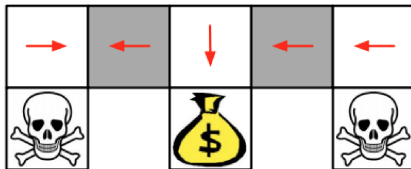
- ▶ Consider a Grid World as in the image
- ▶ The agent can move in four direction (N, E, W, S) if valid
- ▶ The agent **cannot** differentiate the grey states

Example: Aliased Grid world



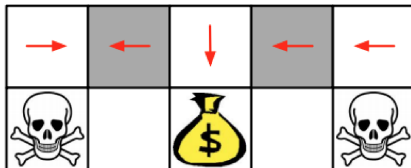
- Under aliasing, an optimal deterministic policy will either
 - move W in both grey states (shown by red arrows)
 - move E in both grey states

Example: Aliased Grid world



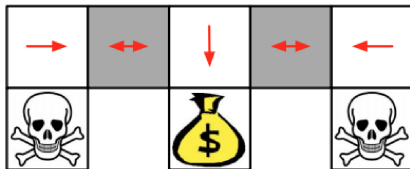
- ▶ Under aliasing, an optimal deterministic policy will either
 - move W in both grey states (shown by red arrows)
 - move E in both grey states
- ▶ Either way, it can get stuck and never reach the money

Example: Aliased Grid world



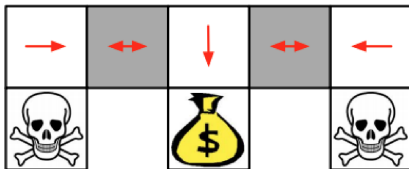
- ▶ Under aliasing, an optimal deterministic policy will either
 - move W in both grey states (shown by red arrows)
 - move E in both grey states
- ▶ Either way, it can get stuck and never reach the money
- ▶ Similarly, Value-based RL learns a near-deterministic policy (e.g., ϵ -greedy)
- ▶ As a result, it will traverse the corridor for a long time depending on the value of Epsilon

Example: Aliased Grid world



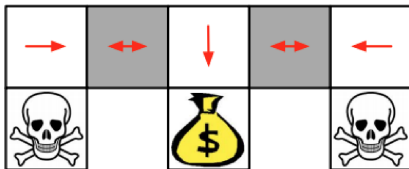
- ▶ An optimal stochastic policy will randomly move E or W in grey states
 - $\pi_{\theta}(\text{wall to N and S, move E}) = 0.5$
 - $\pi_{\theta}(\text{wall to N and S, move W}) = 0.5$

Example: Aliased Grid world



- ▶ An optimal stochastic policy will randomly move E or W in grey states
 - $\pi_{\theta}(\text{wall to N and S, move E}) = 0.5$
 - $\pi_{\theta}(\text{wall to N and S, move W}) = 0.5$
- ▶ It will reach the goal state in a few steps with high probability

Example: Aliased Grid world



- ▶ An optimal stochastic policy will randomly move E or W in grey states
 - $\pi_{\theta}(\text{wall to N and S, move E}) = 0.5$
 - $\pi_{\theta}(\text{wall to N and S, move W}) = 0.5$
- ▶ It will reach the goal state in a few steps with high probability
- ▶ Policy-based RL can learn the optimal stochastic policy

Policy Search: **Limitations of policy gradients**

- ▶ Sample efficiency is poor
 - We throw out each batch of data immediately after just one gradient step
 - Why? PG is an on-policy expectation.

- ▶ Sample efficiency is poor
 - We throw out each batch of data immediately after just one gradient step
 - Why? PG is an on-policy expectation.
 - Recycling old data to estimate policy gradients is hard
 - Potential Solution: Use trajectories from other policies with importance sampling.

- ▶ Sample efficiency is poor
 - We throw out each batch of data immediately after just one gradient step
 - Why? PG is an on-policy expectation.
 - Recycling old data to estimate policy gradients is hard
 - Potential Solution: Use trajectories from other policies with importance sampling.
- ▶ Distance in parameter space $\neq$ distance in policy space!

- ▶ Sample efficiency is poor
 - We throw out each batch of data immediately after just one gradient step
 - Why? PG is an on-policy expectation.
 - Recycling old data to estimate policy gradients is hard
 - Potential Solution: Use trajectories from other policies with importance sampling.
- ▶ Distance in parameter space $\neq$ distance in policy space!
 - What is policy space? For tabular case, set of matrices

$$\Pi = \left\{ \pi : \pi \in \mathbb{R}^{|S| \times |A|}, \pi_{sa} \geq 0 \right\}$$

- ▶ Sample efficiency is poor
 - We throw out each batch of data immediately after just one gradient step
 - Why? PG is an on-policy expectation.
 - Recycling old data to estimate policy gradients is hard
 - Potential Solution: Use trajectories from other policies with importance sampling.
- ▶ Distance in parameter space $\neq$ distance in policy space!
 - What is policy space? For tabular case, set of matrices

$$\Pi = \left\{ \pi : \pi \in \mathbb{R}^{|S| \times |A|}, \pi_{sa} \geq 0 \right\}$$

- Policy gradients take steps in parameter space
- Step size is hard to get right as a result

Policy Search: **Importance Sampling**

- Importance sampling is a technique for estimating expectations using samples drawn from a different distribution.

$$\begin{aligned} E_{x \sim P}[f(x)] &= \int P(x) f(x) dx \\ &= \int P(x) \frac{Q(x)}{Q(x)} f(x) dx \\ &= \int Q(x) \frac{P(x)}{Q(x)} f(x) dx \\ &= E_{x \sim Q} \left[\frac{P(x)}{Q(x)} f(x) \right] \end{aligned}$$

$$\therefore E_{x \sim P}[f(x)] = E_{x \sim Q} \left[\frac{P(x)}{Q(x)} f(x) \right] \approx \frac{1}{|D|} \sum_{x \in D} \frac{P(x)}{Q(x)} f(x)$$

- The ratio $P(x)/Q(x)$ is the importance sampling weight for x

$$\therefore E_{x \sim P}[f(x)] = E_{x \sim Q} \left[\frac{P(x)}{Q(x)} f(x) \right] \approx \frac{1}{|D|} \sum_{x \in D} \frac{P(x)}{Q(x)} f(x)$$

- ▶ The ratio $P(x)/Q(x)$ is the importance sampling weight for x
- ▶ What is the variance of an importance sampling estimator?

$$\begin{aligned} \text{var}(\hat{\mu}_Q) &= \frac{1}{N} \text{var} \left(\frac{P(x)}{Q(x)} f(x) \right) \\ &= \frac{1}{N} \left(E_{x \sim Q} \left[\left(\frac{P(x)}{Q(x)} f(x) \right)^2 \right] - E_{x \sim Q} \left[\frac{P(x)}{Q(x)} f(x) \right]^2 \right) \\ &= \frac{1}{N} \left(E_{x \sim P} \left[\frac{P(x)}{Q(x)} f(x)^2 \right] - E_{x \sim P} [f(x)]^2 \right) \end{aligned}$$

$$\therefore E_{x \sim P}[f(x)] = E_{x \sim Q} \left[\frac{P(x)}{Q(x)} f(x) \right] \approx \frac{1}{|D|} \sum_{x \in D} \frac{P(x)}{Q(x)} f(x)$$

- ▶ The ratio $P(x)/Q(x)$ is the importance sampling weight for x
- ▶ What is the variance of an importance sampling estimator?

$$\begin{aligned} \text{var}(\hat{\mu}_Q) &= \frac{1}{N} \text{var} \left(\frac{P(x)}{Q(x)} f(x) \right) \\ &= \frac{1}{N} \left(E_{x \sim Q} \left[\left(\frac{P(x)}{Q(x)} f(x) \right)^2 \right] - E_{x \sim Q} \left[\frac{P(x)}{Q(x)} f(x) \right]^2 \right) \\ &= \frac{1}{N} \left(E_{x \sim P} \left[\frac{P(x)}{Q(x)} f(x)^2 \right] - E_{x \sim P} [f(x)]^2 \right) \end{aligned}$$

- ▶ The term in red is problematic - if $\frac{P(x)}{Q(x)}$ is large in the wrong places, the variance of the estimator explodes.

- Now, let's put this in policy gradient. $\pi_{\theta'}$ represents new policy.

$$\begin{aligned}\nabla_{\theta'} \mathcal{J}(\theta') &= E_{\tau \sim \pi_{\theta'}} \left[\sum_{t \geq 0} \gamma^t \nabla_{\theta'} \log \pi_{\theta'}(a_t | s_t) A^{\pi_{\theta'}}(s_t, a_t) \right] \\ &= E_{\tau \sim \pi_{\theta}} \left[\sum_{t \geq 0} \frac{P(\tau_t | \pi_{\theta'})}{P(\tau_t | \pi_{\theta})} \gamma^t \nabla_{\theta'} \log \pi_{\theta'}(a_t | s_t) A^{\pi_{\theta'}}(s_t, a_t) \right]\end{aligned}$$

- Now, let's put this in policy gradient. $\pi_{\theta'}$ represents new policy.

$$\begin{aligned}\nabla_{\theta'} \mathcal{J}(\theta') &= E_{\tau \sim \pi_{\theta'}} \left[\sum_{t \geq 0} \gamma^t \nabla_{\theta'} \log \pi_{\theta'}(a_t | s_t) A^{\pi_{\theta'}}(s_t, a_t) \right] \\ &= E_{\tau \sim \pi_{\theta}} \left[\sum_{t \geq 0} \frac{P(\tau_t | \pi_{\theta'})}{P(\tau_t | \pi_{\theta})} \gamma^t \nabla_{\theta'} \log \pi_{\theta'}(a_t | s_t) A^{\pi_{\theta'}}(s_t, a_t) \right]\end{aligned}$$

$$\frac{P(\tau_t | \pi_{\theta'})}{P(\tau_t | \pi_{\theta})} = \frac{\mu(s_0) \prod_{t'=0}^t P(s_{t'+1} | s_{t'}, a_{t'}) \pi_{\theta'}(s_{t'}, a_{t'})}{\mu(s_0) \prod_{t'=0}^t P(s_{t'+1} | s_{t'}, a_{t'}) \pi_{\theta}(s_{t'}, a_{t'})} = \prod_{t'=0}^t \frac{\pi_{\theta'}(s_{t'}, a_{t'})}{\pi_{\theta}(s_{t'}, a_{t'})}$$

- Now, let's put this in policy gradient. $\pi_{\theta'}$ represents new policy.

$$\begin{aligned}\nabla_{\theta'} \mathcal{J}(\theta') &= E_{\tau \sim \pi_{\theta'}} \left[\sum_{t \geq 0} \gamma^t \nabla_{\theta'} \log \pi_{\theta'}(a_t | s_t) A^{\pi_{\theta'}}(s_t, a_t) \right] \\ &= E_{\tau \sim \pi_{\theta}} \left[\sum_{t \geq 0} \frac{P(\tau_t | \pi_{\theta'})}{P(\tau_t | \pi_{\theta})} \gamma^t \nabla_{\theta'} \log \pi_{\theta'}(a_t | s_t) A^{\pi_{\theta'}}(s_t, a_t) \right]\end{aligned}$$

$$\frac{P(\tau_t | \pi_{\theta'})}{P(\tau_t | \pi_{\theta})} = \frac{\mu(s_0) \prod_{t'=0}^t P(s_{t'+1} | s_{t'}, a_{t'}) \pi_{\theta'}(s_{t'}, a_{t'})}{\mu(s_0) \prod_{t'=0}^t P(s_{t'+1} | s_{t'}, a_{t'}) \pi_{\theta}(s_{t'}, a_{t'})} = \prod_{t'=0}^t \frac{\pi_{\theta'}(s_{t'}, a_{t'})}{\pi_{\theta}(s_{t'}, a_{t'})}$$

- Looks useful - what's the issue?

- Now, let's put this in policy gradient. $\pi_{\theta'}$ represents new policy.

$$\begin{aligned}\nabla_{\theta'} \mathcal{J}(\theta') &= E_{\tau \sim \pi_{\theta'}} \left[\sum_{t \geq 0} \gamma^t \nabla_{\theta'} \log \pi_{\theta'}(a_t | s_t) A^{\pi_{\theta'}}(s_t, a_t) \right] \\ &= E_{\tau \sim \pi_{\theta}} \left[\sum_{t \geq 0} \frac{P(\tau_t | \pi_{\theta'})}{P(\tau_t | \pi_{\theta})} \gamma^t \nabla_{\theta'} \log \pi_{\theta'}(a_t | s_t) A^{\pi_{\theta'}}(s_t, a_t) \right]\end{aligned}$$

$$\frac{P(\tau_t | \pi_{\theta'})}{P(\tau_t | \pi_{\theta})} = \frac{\mu(s_0) \prod_{t'=0}^t P(s_{t'+1} | s_{t'}, a_{t'}) \pi_{\theta'}(s_{t'}, a_{t'})}{\mu(s_0) \prod_{t'=0}^t P(s_{t'+1} | s_{t'}, a_{t'}) \pi_{\theta}(s_{t'}, a_{t'})} = \prod_{t'=0}^t \frac{\pi_{\theta'}(s_{t'}, a_{t'})}{\pi_{\theta}(s_{t'}, a_{t'})}$$

- Looks useful - what's the issue?
- Exploding or vanishing importance sampling weights. Even for policies only slightly different from each other, many small differences multiply to become a big difference.

► Solution?

- ▶ Solution?
- ▶ Stay close to the previous policy!
- ▶ We can use KL divergence for that.
- ▶ What is KL-divergence between policies?

$$D_{KL}(\pi' || \pi)[s] = \sum_{a \in A} \pi'(a|s) \log \frac{\pi'(a|s)}{\pi(a|s)}$$

- ▶ Solution?
- ▶ Stay close to the previous policy!
- ▶ We can use KL divergence for that.
- ▶ What is KL-divergence between policies?

$$D_{KL}(\pi' || \pi)[s] = \sum_{a \in A} \pi'(a|s) \log \frac{\pi'(a|s)}{\pi(a|s)}$$

- ▶ Now, we have

$$\nabla_{\theta'} \mathcal{J}(\theta') \text{ s.t. } D_{KL}(\pi' || \pi) \leq \epsilon$$

- ▶ But, recall that for $\nabla_{\theta'} \mathcal{J}(\theta')$ we will still have to compute $\log \pi_{\theta'}(a_t|s_t)A^{\pi_{\theta'}}(s_t, a_t)$ based on current policy.
- ▶ This is not desirable.

- ▶ But, recall that for $\nabla_{\theta'} \mathcal{J}(\theta')$ we will still have to compute $\log \pi_{\theta'}(a_t|s_t)A^{\pi_{\theta'}}(s_t, a_t)$ based on current policy.
- ▶ This is not desirable.
- ▶ So, we make use of Relative Policy Performance Identity. This states that for two policies, $\pi_{\theta'}$ and π_{θ}

$$\mathcal{J}(\pi_{\theta'}) - \mathcal{J}(\pi_{\theta}) = E_{\tau \sim \pi_{\theta'}} \left[\sum_{t=0}^T \gamma^t A^{\pi_{\theta}}(s_t, a_t) \right]$$

- ▶ But, recall that for $\nabla_{\theta'} \mathcal{J}(\theta')$ we will still have to compute $\log \pi_{\theta'}(a_t|s_t) A^{\pi_{\theta'}}(s_t, a_t)$ based on current policy.
- ▶ This is not desirable.
- ▶ So, we make use of Relative Policy Performance Identity. This states that for two policies, $\pi_{\theta'}$ and π_{θ}

$$\mathcal{J}(\pi_{\theta'}) - \mathcal{J}(\pi_{\theta}) = E_{\tau \sim \pi_{\theta'}} \left[\sum_{t=0}^T \gamma^t A^{\pi_{\theta}}(s_t, a_t) \right]$$

- ▶ Using importance sampling, we get

$$\mathcal{J}(\pi_{\theta'}) - \mathcal{J}(\pi_{\theta}) = E_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^T \frac{\pi_{\theta'}(s_t, a_t)}{\pi_{\theta}(s_t, a_t)} \gamma^t A^{\pi_{\theta}}(s_t, a_t) \right]$$

- Can we use this for policy improvement?

- ▶ Can we use this for policy improvement?
- ▶ Recall that our objective is to have policy with maximum return.
- ▶ Basically, we want to

$$\max_{\theta'} \mathcal{J}(\pi_{\theta'})$$

- ▶ Can we use this for policy improvement?
- ▶ Recall that our objective is to have policy with maximum return.
- ▶ Basically, we want to

$$\max_{\theta'} \mathcal{J}(\pi_{\theta'})$$

- ▶ But, this is essentially the same as

$$\max_{\theta'} (\mathcal{J}(\pi_{\theta'}) - \mathcal{J}(\pi_{\theta}))$$

- ▶ Can we use this for policy improvement?
- ▶ Recall that our objective is to have policy with maximum return.
- ▶ Basically, we want to

$$\max_{\theta'} \mathcal{J}(\pi_{\theta'})$$

- ▶ But, this is essentially the same as

$$\max_{\theta'} (\mathcal{J}(\pi_{\theta'}) - \mathcal{J}(\pi_{\theta}))$$

- ▶ Therefore, we can use this as our loss function

$$\mathcal{L}_{\theta'}(\pi_{\theta'}) = \mathcal{J}(\pi_{\theta'}) - \mathcal{J}(\pi_{\theta})$$

Choosing a Step Size for Policy Gradients

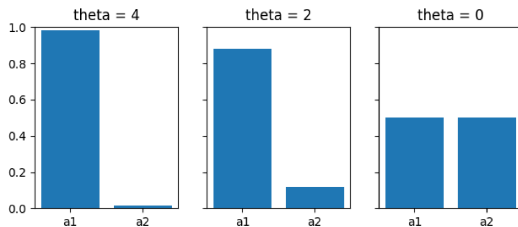
- ▶ But, the problem is more than step size
- ▶ Distance in parameter space $\neq$ distance in policy space!
- ▶ Small changes in the policy parameters can unexpectedly lead to big changes in the policy.

- ▶ But, the problem is more than step size
- ▶ Distance in parameter space $\neq$ distance in policy space!
- ▶ Small changes in the policy parameters can unexpectedly lead to big changes in the policy.
- ▶ Consider a family of policies with parametrization:

$$\pi_{\theta}(a) = \begin{cases} \sigma(\theta) & a = 1 \\ 1 - \sigma(\theta) & a = 2 \end{cases}$$

- ▶ But, the problem is more than step size
- ▶ Distance in parameter space $\neq$ distance in policy space!
- ▶ Small changes in the policy parameters can unexpectedly lead to big changes in the policy.
- ▶ Consider a family of policies with parametrization:

$$\pi_{\theta}(a) = \begin{cases} \sigma(\theta) & a = 1 \\ 1 - \sigma(\theta) & a = 2 \end{cases}$$





Policy Search: **Trust Region Policy Optimization (TRPO)**

- ▶ TRPO updates policies by taking the largest step possible to improve performance, while satisfying a special constraint on how close the new and old policies are allowed to be.
- ▶ The constraint is expressed in terms of KL-Divergence
- ▶ This is different from normal policy gradient, which keeps new and old policies close in parameter space
- ▶ TRPO nicely avoids this kind of collapse, and tends to quickly and monotonically improve performance
- ▶ TRPO uses conjugate gradients for computing the hessian matrix for KL divergence derivative

⁰<https://spinningup.openai.com/en/latest/algorithms/trpo.html>

- ▶ TRPO uses backtracking line search with exponential decay (decay coeff $\alpha \in (0, 1)$, budget L) to make appropriate step sizes

Algorithm 2 Line Search for TRPO

Compute proposed policy step $\Delta_k = \sqrt{\frac{2\delta}{\hat{\mathbf{g}}_k^T \hat{\mathbf{H}}_k^{-1} \hat{\mathbf{g}}_k}} \hat{\mathbf{H}}_k^{-1} \hat{\mathbf{g}}_k$

for $j = 0, 1, 2, \dots, L$ **do**

 Compute proposed update $\theta = \theta_k + \alpha^j \Delta_k$

if $\mathcal{L}_{\theta_k}(\theta) \geq 0$ and $\bar{D}_{KL}(\theta || \theta_k) \leq \delta$ **then**

 accept the update and set $\theta_{k+1} = \theta_k + \alpha^j \Delta_k$

 break

end if

end for

Algorithm 3 Trust Region Policy Optimization

Input: initial policy parameters θ_0

for $k = 0, 1, 2, \dots$ **do**

Collect set of trajectories $\mathcal{D}_k$ on policy $\pi_k = \pi(\theta_k)$

Estimate advantages $\hat{A}_t^{\pi_k}$ using any advantage estimation algorithm

Form sample estimates for

- policy gradient $\hat{g}_k$ (using advantage estimates)
- and KL-divergence Hessian-vector product function $f(v) = \hat{H}_k v$

Use CG with n_{cg} iterations to obtain $x_k \approx \hat{H}_k^{-1} \hat{g}_k$

Estimate proposed step $\Delta_k \approx \sqrt{\frac{2\delta}{x_k^T \hat{H}_k x_k}} x_k$

Perform backtracking line search with exponential decay to obtain final update

$$\theta_{k+1} = \theta_k + \alpha^j \Delta_k$$

end for

Policy Search: **Proximal Policy Optimization (PPO)**

- ▶ PPO is motivated by the same question as TRPO: how can we take the biggest possible improvement step on a policy using the data we currently have, without stepping so far that we accidentally cause performance collapse?
- ▶ Where TRPO tries to solve this problem with a complex second-order method, PPO is a family of first-order methods that use a few other tricks to keep new policies close to old.
- ▶ It approximately enforce KL constraint without computing natural gradients.
- ▶ PPO methods are significantly simpler to implement, and empirically seem to perform at least as well as TRPO.
- ▶ There are two primary variants of PPO: PPO-Penalty and PPO-Clip.

⁰<https://spinningup.openai.com/en/latest/algorithms/ppo.html>

► Adaptive KL Penalty

- Policy update solves unconstrained optimization problem

$$\theta_{k+1} = \arg \max_{\theta} \mathcal{L}_{\theta_k}(\theta) - \beta_k \overline{D}_{KL}(\theta || \theta_k)$$

- Penalty coefficient β_k changes between iterations to approximately enforce KL-divergence constraint

► Adaptive KL Penalty

- Policy update solves unconstrained optimization problem

$$\theta_{k+1} = \arg \max_{\theta} \mathcal{L}_{\theta_k}(\theta) - \beta_k \overline{D}_{KL}(\theta || \theta_k)$$

- Penalty coefficient β_k changes between iterations to approximately enforce KL-divergence constraint

► Clipped Objective

- New objective function: let $r_t(\theta) = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_k}(a_t | s_t)}$. Then,

$$\mathcal{L}_{\theta_k}^{CLIP} = E_{\tau \sim \pi_k} \left[\sum_{t=0}^T \left[r_t(\theta) \hat{A}_t^{\pi_k}, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t^{\pi_k} \right] \right]$$

- ϵ is a hyperparameter (e.g., $\epsilon = 0.2$)
- Policy update is

$$\theta_{k+1} = \arg \max_{\theta} \mathcal{L}_{\theta_k}^{CLIP}$$

Algorithm 4 PPO with Adaptive KL Penalty

Input: initial policy parameters θ_0 , initial KL penalty β_0 , target KL-divergence δ

for $k = 0, 1, 2, \dots$ **do**

Collect set of partial trajectories $\mathcal{D}_k$ on policy $\pi_k = \pi(\theta_k)$

Estimate advantages $\hat{A}_t^{\pi_k}$ using any advantage estimation algorithm

Compute policy update

$$\theta_{k+1} = \arg \max_{\theta} \mathcal{L}_{\theta_k}(\theta) - \beta_k \bar{D}_{KL}(\theta || \theta_k)$$

by taking K steps of minibatch SGD (via Adam)

if $\bar{D}_{KL}(\theta_{k+1} || \theta_k) \geq 1.5\delta$ **then**

$$\beta_{k+1} = 2\beta_k$$

else if $\bar{D}_{KL}(\theta_{k+1} || \theta_k) \leq \delta/1.5$ **then**

$$\beta_{k+1} = \beta_k/2$$

end if

end for

- PPO clip is more widely used as it seems to work at least as well as PPO with KL penalty, but is simpler to implement

Algorithm 5 PPO with Clipped Objective

Input: initial policy parameters θ_0 , clipping threshold ϵ

for $k = 0, 1, 2, \dots$ **do**

Collect set of partial trajectories $\mathcal{D}_k$ on policy $\pi_k = \pi(\theta_k)$

Estimate advantages $\hat{A}_t^{\pi_k}$ using any advantage estimation algorithm

Compute policy update

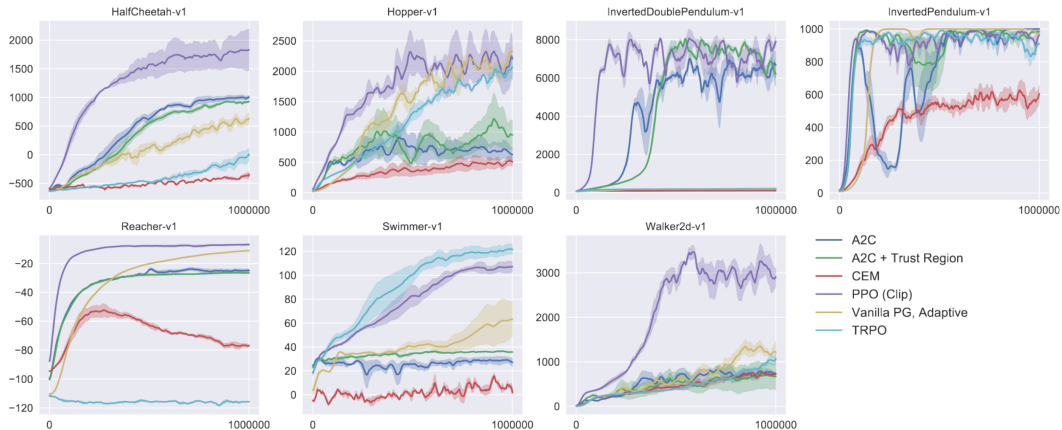
$$\theta_{k+1} = \arg \max_{\theta} \mathcal{L}_{\theta_k}^{CLIP}(\theta)$$

by taking K steps of minibatch SGD (via Adam), where

$$\mathcal{L}_{\theta_k}^{CLIP}(\theta) = \mathbb{E}_{\tau \sim \pi_k} \left[\sum_{t=0}^T \left[\min(r_t(\theta) \hat{A}_t^{\pi_k}, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t^{\pi_k}) \right] \right]$$

end for

Empirical Performance of PPO



⁰Schulman, Wolski, Dhariwal, Radford, Klimov, 2017

Policy Search: **Summary**

Feature	TRPO	PPO
Optimization	Constrained (KL)	Unconstrained (clipped)
Implementation	Complex	Simple
Performance	Strong	Comparable
Speed	Slower	Faster (SGD-friendly)
Used in	Robotics, theory	Industry, games

- ▶ Both methods remain sensitive to reward scaling, exploration strategies, and advantage estimation quality.
- ▶ PPO's clipping mechanism is heuristic and may under- or over-constrain policy updates.
- ▶ Both can struggle in environments with very sparse rewards.
- ▶ Stability and convergence are not guaranteed in general MDPs.

- ▶ In some cases, learning a stochastic policy is preferable to a deterministic policy.
- ▶ Policy gradient methods often suffer from poor sample efficiency.
- ▶ Importance sampling can help improve sample efficiency.
- ▶ However, it is important to ensure that the current policy is not too different from the policy used to collect trajectories.
- ▶ Small changes in policy parameters can sometimes lead to large, unexpected changes in the policy.
- ▶ TRPO uses importance sampling to take multiple gradient steps and constrains the optimization objective in policy space.
- ▶ PPO achieves similar goals by approximately enforcing a KL-divergence constraint without computing natural gradients.

Policy Search: **References**

- [1] Schulman, J., Levine, S., Abbeel, P., Jordan, M., & Moritz, P. (2015).
Trust Region Policy Optimization.
In Proceedings of the 32nd International Conference on Machine Learning (ICML).
- [2] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017).
Proximal Policy Optimization Algorithms.
arXiv preprint arXiv:1707.06347.
- [3] Andrychowicz, M., Baker, B., Chociej, M., et al. (2020).
What Matters for On-Policy Deep RL?
arXiv preprint arXiv:2006.05990.

- [4] Haarnoja, T., Zhou, A., Abbeel, P., & Levine, S. (2018).

Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor.

In *Proceedings of the 35th International Conference on Machine Learning (ICML)*.

- [5] OpenAI Baselines.

<https://github.com/openai/baselines>

- [6] Spinning Up by OpenAI.

<https://spinningup.openai.com>

- [7] Sergey Levine, Berkeley CS285: Deep Reinforcement Learning.

<https://rail.eecs.berkeley.edu/deeprlcourse-fa21/>

- [8] Chelsea Finn & Karol Hausman, Stanford CS224R: Deep Reinforcement Learning.
<http://cs224r.stanford.edu/>

- [9] Emma Brunskill, Stanford CS234: Reinforcement Learning.
<https://web.stanford.edu/class/cs234/>

- [10] Joshua Achiam, Berkeley CS294: Deep Reinforcement Learning.
<http://rail.eecs.berkeley.edu/deeprlcourse-fa17/>